第五次实验课主要内容

# 文件 I/O 操作

**实验 1　教材“思考题”遴选（第 5 章）**

**1.　实验目的**

掌握如何利用结构体 struct stat 以及 stat 函数、fstat 函数以及 lstat 函数判断 Linux 文件的类型、权限及其它属性。

**2.　实验任务与要求**

参考 ppt 讲义“第 5 章文件 I/O 操作”，设计程序，判断指定文件的类型、权限以及最后一次的访问和修改时间。其中指定文件可以作为程序的运行参数在命令行指定，并可以是任意类型的文件（例如普通文件、目录、字符设备文件、块设备文件、链接文件等）。参考如下的程序运行结果完成该题：

提示：main 函数使用形式参数，例如 int main(int argc, char *argv[])

```
root@cg:~/Desktop/exp5# vim pro2.c
root@cg:~/Desktop/exp5# vim q1.c
root@cg:~/Desktop/exp5# ./pro2 q1.c
bash: ./pro2: No such file or directory
root@cg:~/Desktop/exp5# gcc -o q1 q1.c
root@cg:~/Desktop/exp5# gcc -o pro2 pro2.c
root@cg:~/Desktop/exp5# ./pro2 q1.c
普通文件文件 q1.c 的类型及权限是：-rw-r--r--
该文件被最后一次访问的时间是：Wed Apr 22 11:51
:59 2026
该文件被最后一次修改的时间是：Wed Apr 22 11:51
:29 2026
root@cg:~/Desktop/exp5# ./pro2 /dev/tty
字符设备文件文件 /dev/tty 的类型及权限是：crw-rw-rw-
该文件被最后一次访问的时间是：Wed Apr 22 11:25:43 2026
该文件被最后一次修改的时间是：Wed Apr 22 11:25:43 2026
root@cg:~/Desktop/exp5# ./pro2 ~^C
root@cg:~/Desktop/exp5# ./pro2 ~/Desktop/exp5
目录文件 /headless/Desktop/exp5 的类型及权限是：drwxr-xr-x
该文件被最后一次访问的时间是：Wed Apr 22 11:49:10 2026
该文件被最后一次修改的时间是：Wed Apr 22 11:52:14 2026
```

```c
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>
#include <time.h>
int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("用法：%s <文件名>\n", argv[0]);
        return 1;
    }
    struct stat buf;
    if (lstat(argv[1], &buf) == -1) {
        perror("lstat 执行错误");
        return 1;
    }
    // 解析文件类型
    if (S_ISREG(buf.st_mode))   printf("普通文件");
    else if (S_ISDIR(buf.st_mode)) printf("目录");
    else if (S_ISCHR(buf.st_mode)) printf("字符设备文件");
    else if (S_ISBLK(buf.st_mode)) printf("块设备文件");
    else if (S_ISFIFO(buf.st_mode)) printf("FIFO管道文件");
    else if (S_ISLNK(buf.st_mode)) printf("符号链接文件");
    else if (S_ISSOCK(buf.st_mode)) printf("套接字文件");
    else printf("未知类型");
    // 解析权限位
    char perm[11] = {0};
    perm[0] = (S_ISREG(buf.st_mode))   ? '-' :
              (S_ISDIR(buf.st_mode)) ? 'd' :
              (S_ISCHR(buf.st_mode)) ? 'c' :
              (S_ISBLK(buf.st_mode)) ? 'b' :
              (S_ISFIFO(buf.st_mode)) ? 'p' :
              (S_ISLNK(buf.st_mode)) ? 'l' :
              (S_ISSOCK(buf.st_mode)) ? 's' : '?';
    perm[1] = (buf.st_mode & S_IRUSR) ? 'r' : '-';
    perm[2] = (buf.st_mode & S_IWUSR) ? 'w' : '-';
    perm[3] = (buf.st_mode & S_IXUSR) ? 'x' : '-';
    perm[4] = (buf.st_mode & S_IRGRP) ? 'r' : '-';
```


```c
    perm[1] = (buf.st_mode & S_IRUSR) ? 'r' : '-';
    perm[2] = (buf.st_mode & S_IWUSR) ? 'w' : '-';
    perm[3] = (buf.st_mode & S_IXUSR) ? 'x' : '-';
    perm[4] = (buf.st_mode & S_IRGRP) ? 'r' : '-';
```


```c
    perm[5] = (buf.st_mode & S_IWGRP) ? 'w' : '-';
    perm[6] = (buf.st_mode & S_IXGRP) ? 'x' : '-';
    perm[7] = (buf.st_mode & S_IROTH) ? 'r' : '-';
    perm[8] = (buf.st_mode & S_IWOTH) ? 'w' : '-';
    perm[9] = (buf.st_mode & S_IXOTH) ? 'x' : '-';
    printf("文件 %s 的类型及权限是：%s\n", argv[1], perm);
    printf("该文件被最后一次访问的时间是：%s", ctime(&buf.st_atime));
    printf("该文件被最后一次修改的时间是：%s", ctime(&buf.st_mtime));
    return 0;
}
```

```
root@cg:~/Desktop/exp5# echo "test" > file
root@cg:~/Desktop/exp5# chmod 755 file
root@cg:~/Desktop/exp5# ln -sf file file.sl
root@cg:~/Desktop/exp5# touch file
root@cg:~/Desktop/exp5# ln -sf file file.sl
root@cg:~/Desktop/exp5# ./pro2 file.sl
符号链接文件文件 file.sl 的类型及权限是：lrwxrwxrwx
该文件被最后一次访问的时间是：Wed Apr 22 12:19:13 2026
该文件被最后一次修改的时间是：Wed Apr 22 12:19:11 2026
root@cg:~/Desktop/exp5# ls -l file
-rwxr-xr-x 1 root root 5 Apr 22 12:18 file
root@cg:~/Desktop/exp5# ls -l file.sl
lrwxrwxrwx 1 root root 4 Apr 22 12:19 file.sl -> file
```

./pro2 file.sl 显示是因为程序使用了 stat()，它返回了 目标文件 file 的信息

而 s -l file.sl 显示是因为 ls -l 使用了 lstat()，它返回了 符号链接本身 file.sl 的信息

只需要/修改./pro2 中 stat（会跟踪符号链接）中的：

```
if (stat(filename, &buf) == -1) {
    perror("stat");
    return 1;
}
```

而改为使用 lstat（不跟踪符号链接）

```
if (lstat(filename, &buf) == -1) {
    perror("lstat");
    return 1;
}
```

即可保持一致

实验体会：

通过本次实验，我深刻理解了 Linux 文件系统中文件属性的获取与解析方法，掌握了使用 stat 和 lstat 函数读取文件元数据的技巧。实验过程中，我学会了如何通过解析 struct stat 结构体中的 st_mode 字段来判断文件类型，并利用位运算提取和显示文件的权限信息。更重要的是，我认识到 stat 与 lstat 函数在处理符号链接时的关键区别：stat 函数会跟踪符号链接并返回目标文件的属性，而 lstat 函数则直接返回链接文件本身的属性。这一区别解释了为何最初程序输出与 ls 命令显示结果不一致，也让我明白了在实际编程中应根据具体需求选择合适的函数。此外，通过将时间戳转换为可读格式，我进一步熟悉了 Linux 时间处理的相关函数。这次实验不仅加深了我对文件 I/O 操作的理解，也提升了我解决实际问题的能力，为后续学习系统编程打下了坚实的基础。

## 实验 2 文件的读写操作

### 1. 实验目的

巩固文件的不带缓冲流的读写方法

**2. 实验任务与要求**

编写程序，将当前目录下的子目录 dir1 下的普通文件 file 拷贝至当前目录下的子目录 dir2 中（先自建 dir1、file、dir2）。要求：不能使用 system 函数或 exec 系列函数。

**3. 编写程序**

给出实现该任务的源代码、运行结果的截图，并写出实验体会（可选）。

```
File  Edit  View  Terminal  Tabs  Help
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <errno.h>
#define BUFFER_SIZE 4096  // 缓冲区大小，4KB
int main() {
    int src_fd, dst_fd;  // 源文件和目标文件的文件描述符
    ssize_t bytes_read, bytes_written;
    char buffer[BUFFER_SIZE];
    // 打开源文件（dir1/file）
    src_fd = open("dir1/file", O_RDONLY);
    if (src_fd == -1) {
        perror("打开源文件失败");
        return 1;
    }
    // 创建并打开目标文件（dir2/file）
    dst_fd = open("dir2/file", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dst_fd == -1) {
        perror("打开/创建目标文件失败");
        close(src_fd);
        return 1;
-- INSERT --                                                    1,6
```

```
File  Edit  View  Terminal  Tabs  Help
    }
    // 读取源文件并写入目标文件
    while ((bytes_read = read(src_fd, buffer, BUFFER_SIZE)) > 0) {
        bytes_written = write(dst_fd, buffer, bytes_read);
        if (bytes_written != bytes_read) {
            perror("写入目标文件失败");
            close(src_fd);
            close(dst_fd);
            return 1;
        }
    }
    if (bytes_read == -1) {
        perror("读取源文件失败");
        close(src_fd);
        close(dst_fd);
        return 1;
    }
    close(src_fd);
    close(dst_fd);
    printf("文件拷贝成功：dir1/file 已拷贝到 dir2/file\n");
    printf("目标文件的权限为：rw-r--r--\n");
    return 0;
}
-- INSERT --                                                    46,2
```

```
root@cg:~/Desktop/exp5# gcc -o q2 q2.c
root@cg:~/Desktop/exp5# ./q2
文件拷贝成功：dir1/file 已拷贝到 dir2/file
目标文件的权限为：rw-r--r--
```

实验体会

本次文件读写操作实验让我深入理解了 Linux 系统下的文件 I/O 编程。通过实现文件拷贝功能，我掌握了不带缓冲流读写的基本方法，包括使用 open、read、write、close 等系统调用。在实验过程中，我注意到缓冲区大小的选择对拷贝效率有显著影响，较大的缓冲区可以减少系统调用次数，提高性能。同时，我学会了正确处理文件操作的错误情况，如检查文件打开是否成功、读取写入是否完整等。通过对比源文件和目标文件的内容、权限和时间戳，我验证了拷贝操作的正确性和完整性。这次实验加深了我对文件系统操作的理解，为后续学习更复杂的文件处理技术奠定了基础。特别值得注意的是，与高级语言的文件操作 API 相比，系统级 I/O 调用提供了更底层的控制，虽然需要更多的手动管理，但也带来了更高的灵活性。

## 实验 3　课后作业题遴选（第 5 章）

### 1.　实验目的

熟悉 Linux 下不带缓存的文件 I/O 操作。

### 2.　实验任务与要求

**第 1 题：**

设计一个程序，要求打开文件“pass”，如果没有该文件，新建该文件，权限设置为文件的所有者具有读写权限；如果该文件存在，则清空其内容。

```
root@cg:~/Desktop/exp5# vim pass_file.c
root@cg:~/Desktop/exp5# gcc -o pass_file pass_file.c
root@cg:~/Desktop/exp5# ./pass_file
文件 'pass' 不存在，已创建新文件，权限设置为rw-------
当前文件 'pass' 的状态：
文件大小：19 字节
文件权限：-rw------- (八进制：0600)
root@cg:~/Desktop/exp5# ./pass_file
文件 'pass' 已存在，已清空其内容
当前文件 'pass' 的状态：
文件大小：0 字节
文件权限：-rw------- (八进制：0600)
root@cg:~/Desktop/exp5# ls -l pass
-rw------- 1 root root 0 Apr 22 12:34 pass
root@cg:~/Desktop/exp5#
```

File Edit View Terminal Tabs Help

```c
#include <stdio.h>

#include <stdlib.h>

#include <fcntl.h>

#include <unistd.h>

#include <sys/stat.h>

int main() {

    int fd;

    char filename[] = "pass";

    // 尝试以读写模式打开文件

    fd = open(filename, O_RDWR);

    if (fd == -1) {

        // 文件不存在，创建新文件

        fd = open(filename, O_RDWR | O_CREAT, 0600);  // 0600表示rw-------

        if (fd == -1) {

            perror("创建文件失败");

            return 1;

        }

        printf("文件 '%s' 不存在，已创建新文件，权限设置为rw-------\n", filename);

        // 可选：写入一些初始内容

        char *init_content = "这是新创建的文件\n";

        write(fd, init_content, sizeof(char) * 19);

    } else {

        // 文件存在，清空内容

        if (ftruncate(fd, 0) == -1) {
-- INSERT --                                                    1,2           To
```

```
        if (ftruncate(fd, 0) == -1) {
            perror("清空文件失败");
            close(fd);
            return 1;
        }
        printf("文件 '%s' 已存在，已清空其内容\n", filename);
    }
    // 显示文件信息
    printf("当前文件 '%s' 的状态:\n", filename);
    // 获取文件状态
    struct stat file_stat;
    if (fstat(fd, &file_stat) == 0) {
        printf("文件大小: %ld 字节\n", (long)file_stat.st_size);
        // 显示权限
        printf("文件权限: ");
        printf((S_ISDIR(file_stat.st_mode)) ? "d" : "-");
        printf((file_stat.st_mode & S_IRUSR) ? "r" : "-");
        printf((file_stat.st_mode & S_IWUSR) ? "w" : "-");
        printf((file_stat.st_mode & S_IXUSR) ? "x" : "-");
        printf((file_stat.st_mode & S_IRGRP) ? "r" : "-");
        printf((file_stat.st_mode & S_IWGRP) ? "w" : "-");
        printf((file_stat.st_mode & S_IXGRP) ? "x" : "-");
        printf((file_stat.st_mode & S_IROTH) ? "r" : "-");
-- INSERT --                                                    46,1
```

```
        printf((file_stat.st_mode & S_IROTH) ? "r" : "-");
        printf((file_stat.st_mode & S_IWOTH) ? "w" : "-");
        printf((file_stat.st_mode & S_IXOTH) ? "x" : "-");
        printf(" (八进制: %04o)\n", file_stat.st_mode & 0777);
    }
    close(fd);
    return 0;
}
-- INSERT --                                                    105,7
```

**第 2 题：**

设计一个程序，要求新建一个文件“hello”，利用 write 函数将“Linux 下 C 程序设计”字符串写入该文件。

```
root@cg:~/Desktop/exp5# vim hello_file.c
root@cg:~/Desktop/exp5# gcc -o hello_file hello_file.c
root@cg:~/Desktop/exp5# ./hello_file
成功写入 22 字节到文件 'hello'
写入内容: Linux下C程序设计

文件信息:
文件名: hello
文件大小: 22 字节
文件权限: 0644 (rw-r--r--)

验证文件内容:
------------------------
Linux下C程序设计
------------------------
root@cg:~/Desktop/exp5# ls -l hello
-rw-r--r-- 1 root root 22 Apr 22 12:37 hello
root@cg:~/Desktop/exp5# cat hello
Linux下C程序设计
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd;
    char filename[] = "hello";
    char *content = "Linux下C程序设计\n";
    ssize_t bytes_written;

    // 创建文件（如果存在则清空），设置权限为0644
    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("创建文件失败");
        return 1;
    }

    // 计算字符串长度
    size_t content_len = strlen(content);

    // 写入内容
    bytes_written = write(fd, content, content_len);
```

-- INSERT -- 1,

```
    if (bytes_written == -1) {
        perror("写入文件失败");
        close(fd);
        return 1;
    }

    printf("成功写入 %zd 字节到文件 '%s'\n", bytes_written, filename);
    printf("写入内容: %s", content);

    // 显示文件信息
    printf("\n文件信息:\n");
    printf("文件名: %s\n", filename);
    printf("文件大小: %zd 字节\n", bytes_written);
    printf("文件权限: 0644 (rw-r--r--)\n");

    // 读取并显示文件内容以验证
    close(fd);  // 先关闭文件

    // 重新以只读方式打开
    fd = open(filename, O_RDONLY);
    if (fd != -1) {
        printf("\n验证文件内容:\n");
        printf("----------------------\n");
```

```
        char buffer[100];
        ssize_t bytes_read;

        while ((bytes_read = read(fd, buffer, sizeof(buffer)-1)) > 0) {
            buffer[bytes_read] = '\0';  // 添加字符串结束符
            printf("%s", buffer);
        }
        printf("-----------------------\n");

        close(fd);
    }

    return 0;
}
-- INSERT --                                                    125,2
```

**第 6 题：（可选）**

写几条 Linux 命令，改变当前终端 umask 的值，然后新建一个目录，使它的初始权限为—x—x—x，即只有可执行权限。

```
root@cg:~/Desktop/exp5# umask
0022
root@cg:~/Desktop/exp5# umask 0666
root@cg:~/Desktop/exp5# mkdir new
root@cg:~/Desktop/exp5# ls -ld new
d--x--x--x 2 root root 4096 Apr 22 12:42 new
```

## 3. 编写程序

给出实现该任务的源代码、运行结果的截图，并写出实验体会（可选）。

实验体会

通过本次实验，我深入掌握了 Linux 下不带缓冲的文件 I/O 操作。open()函数的不同标志位（O_RDWR、O_CREAT、O_TRUNC 等）组合使用，能够灵活控制文件的打开和创建行为。在文件权限设置方面，我理解了八进制权限表示法（如 0600 表示所有者读写权限）以及与 umask 的关系。通过 ftruncate()函数清空文件内容，我学会了如何修改文件大小。write()函数的使用让我熟悉了低级 I/O 的写入操作。实验还让我认识到错误处理在系统编程中的重要性，每次系统调用后都应检查返回值以确保操作成功。这些基础知识为后续学习更复杂的文件操作和系统编程打下了坚实的基础。